LAPORAN PRAKTIKUM
PERTEMUAN 3

Laporan ini dibuat untuk memenuhi tugas mata kuliah Pemrograman Berbasis Platform

Dosen Pengampu:
SUGIYANTO



DIBUAT OLEH

1. Rangga Alvian Putra Pratama (240511031)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH CIREBON
TAHUN 2026

Kampus Utama JL Watubelah No. 40 Sumber No. Telp. (0231) 209608

BAB I PENDAHULUAN

1.1 Latar Belakang

Perangkat lunak modern dijalankan pada beragam lingkungan eksekusi: peramban web, server, dan komputer desktop. Masing-masing lingkungan menetapkan cara kode ditulis, diterjemahkan, dan dijalankan. Seorang pengembang yang memahami perbedaan tersebut dapat memilih teknologi yang sesuai dengan kebutuhan proyek, bukan sekadar mengikuti kebiasaan.

Langkah pertama sebelum mempelajari perbedaan itu adalah memastikan perangkat kerja berfungsi. Program "Hello World" digunakan sebagai uji sederhana: bila program kecil ini dapat ditulis, dijalankan, dan menghasilkan keluaran yang diharapkan, maka editor, penerjemah (interpreter), kompilator, dan terminal dapat dianggap terpasang dengan benar. Praktikum ini menerapkan uji tersebut pada tiga platform, yaitu HTML pada peramban, JavaScript pada Node.js, dan Java pada Java Virtual Machine (JVM).

Selain itu, praktikum ini memperkenalkan Git sebagai sistem kontrol versi dan GitHub sebagai layanan penyimpanan repositori jarak jauh. Dengan Git, setiap perubahan berkas tercatat sebagai commit sehingga riwayat pekerjaan dapat ditelusuri dan dipulihkan bila terjadi kesalahan.

1.2 Tujuan

1. Memasang dan memverifikasi perangkat pendukung pengembangan, yaitu Visual Studio Code, Node.js dan npm, JDK, serta Git.
2. Membuat dan menjalankan program Hello World pada platform Web (HTML), Node.js (JavaScript), dan Desktop (Java).
3. Membandingkan alur eksekusi pada ketiga platform: dirender oleh peramban, dijalankan oleh mesin V8, dan dikompilasi menjadi bytecode lalu dijalankan JVM.
4. Menggunakan Git untuk mencatat perubahan secara lokal dan mengunggahnya ke repositori GitHub.
5. Mendokumentasikan kendala yang dijumpai selama praktikum beserta cara penyelesaiannya.

1.3 Tinjauan Pustaka

Platform dan lingkungan eksekusi. Platform dipahami sebagai gabungan perangkat keras, sistem operasi, dan lingkungan runtime yang menyediakan antarmuka pemrograman bagi aplikasi (Sommerville, 2016). Pilihan platform memengaruhi bahasa, perkakas, dan cara pengujian aplikasi.

Platform web. Halaman web disusun dengan HTML sebagai struktur, CSS sebagai tampilan, dan JavaScript sebagai perilaku. Peramban membaca dokumen HTML, membentuk pohon DOM, lalu menampilkannya tanpa langkah kompilasi yang dilakukan pengembang (MDN Web Docs, n.d.).

Node.js. Node.js adalah runtime JavaScript di luar peramban yang memakai mesin V8 dan model masukan/keluaran asinkron berbasis event (Tilkov & Vinoski, 2010; Node.js, n.d.).

Karena tidak berjalan di peramban, objek seperti window dan document tidak tersedia; sebagai gantinya tersedia modul sistem seperti os dan fs.

Java dan JVM. Kode sumber Java (.java) dikompilasi oleh javac menjadi bytecode (.class), kemudian bytecode dieksekusi oleh JVM sehingga program yang sama dapat dijalankan pada sistem operasi berbeda selama JVM tersedia (Lindholm dkk., 2022; Oracle, n.d.). JDK menyediakan kompilator, JVM, dan pustaka standar.

Git dan GitHub. Git adalah sistem kontrol versi terdistribusi yang menyimpan riwayat proyek sebagai rangkaian snapshot (Chacon & Straub, 2014). Berkas yang diubah dipindahkan ke staging area dengan git add, dicatat dengan git commit, dan disinkronkan ke repositori jarak jauh dengan git push atau git pull. GitHub menyediakan hosting repositori tersebut beserta fitur kolaborasi (GitHub Docs, n.d.).

Visual Studio Code dan npm. Visual Studio Code adalah editor kode yang dapat diperluas dengan ekstensi dan memiliki terminal terpadu (Microsoft, n.d.). npm adalah pengelola paket untuk ekosistem Node.js (npm Docs, n.d.).

BAB II ALAT DAN BAHAN

2.1 Alat

Tabel 2.1. Daftar Alat

No.	Alat	Fungsi dalam praktikum
1	Laptop Axioo Pongo 750 Windows 11 Pro	Menulis, menjalankan, dan menguji program
2	Visual Studio Code	Menulis berkas .html, .js, dan .java
3	Peramban web Brave	Menampilkan hasil halaman HTML
4	Terminal (PowerShell/Git Bash)	Menjalankan node, javac, java, dan perintah Git

2.2 Bahan

Tabel 2.2. Daftar Bahan

No.	Bahan	Keterangan
1	Node.js dan npm	Menjalankan skrip JavaScript di terminal
2	JDK 27.0.1.0.3	Mengompilasi dan menjalankan program Java
3	Git	Mencatat versi berkas secara lokal
4	Akun dan repositori https://github.com/ranggaalvian/pbp-240511031	Menyimpan hasil kerja di server jarak jauh
5	Berkas sumber hello.html, hello.js, HelloDesktop.java	Program uji untuk masing-masing platform

BAB III PROSEDUR KERJA

3.1 Verifikasi Perangkat dan Persiapan Folder

1. Buka terminal pada Visual Studio Code, lalu buat folder kerja dan masuk ke dalamnya:

```
mkdir pbp-pertemuan1
cd pbp-pertemuan1
```

2. Periksa versi seluruh perangkat:

```
git --version
node -v
npm -v
javac -version
```

3.2 Program pada Platform Web

1. Buat berkas hello.html.
2. Isi dengan kode berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Praktikum PBP 1</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 2rem; }
    h1 { color: #1a5276; }
  </style>
</head>
<body>
  <h1>Halo dari Platform Web</h1>
  <p>Praktikan: Rangga Alvian Putra Pratama</p>
  <p>NIM: [isi NIM]</p>
</body>
</html>
```

3. Simpan, lalu buka berkas di peramban dan periksa tampilannya.

3.3 Program pada Node.js

1. Buat berkas hello.js.
2. Isi dengan kode berikut:

```
const os = require("os");

console.log("Halo dari Node.js");
```

```
console.log("Versi Node.js : " + process.version);
console.log("Sistem operasi: " + os.platform());
```

3. Jalankan dengan perintah node hello.js dan catat keluaran di terminal.

3.4 Program pada Java Desktop

1. Buat berkas HelloDesktop.java. Nama kelas publik harus sama dengan nama berkas.
2. Isi dengan kode berikut:

```
public class HelloDesktop {
   public static void main(String[] args) {
      System.out.println("Halo dari Java Desktop");
      System.out.println("Versi Java: " + System.getProperty("java.version"));
   }
}
```

3. Kompilasi dengan javac HelloDesktop.java; perintah ini menghasilkan berkas HelloDesktop.class.
4. Jalankan dengan java HelloDesktop dan catat keluarannya.

3.5 Penggunaan Git dan GitHub

1. Buat repositori kosong di GitHub, lalu salin URL-nya.
2. Inisialisasi dan catat perubahan di folder kerja:

```
git init
git add .
git commit -m "Pertemuan 1: hello world tiga platform"
git branch -M main
```

3. Hubungkan dengan repositori jarak jauh lalu unggah:

```
git remote add origin  https://github.com/ranggaalvian/pbp-240511031
git push -u origin main
```

4. Periksa status dan riwayat dengan git status dan git log --oneline, lalu pastikan commit tampil pada halaman GitHub.
5. Bila terjadi kendala sinkronisasi (misalnya remote berisi commit yang belum ada di lokal), jalankan git pull --rebase origin main kemudian ulangi git push.

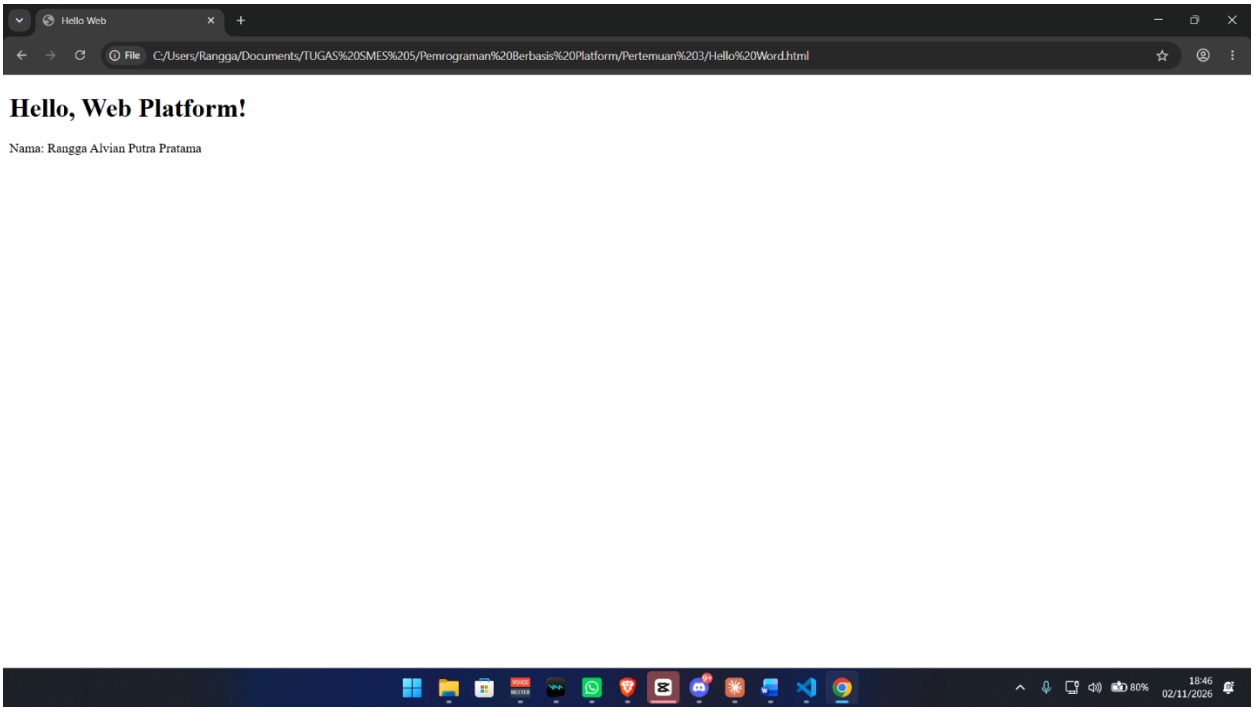
BAB IV HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Tabel 4.1. Rekap hasil pengujian (kolom Hasil Aktual diisi dari praktik Anda)

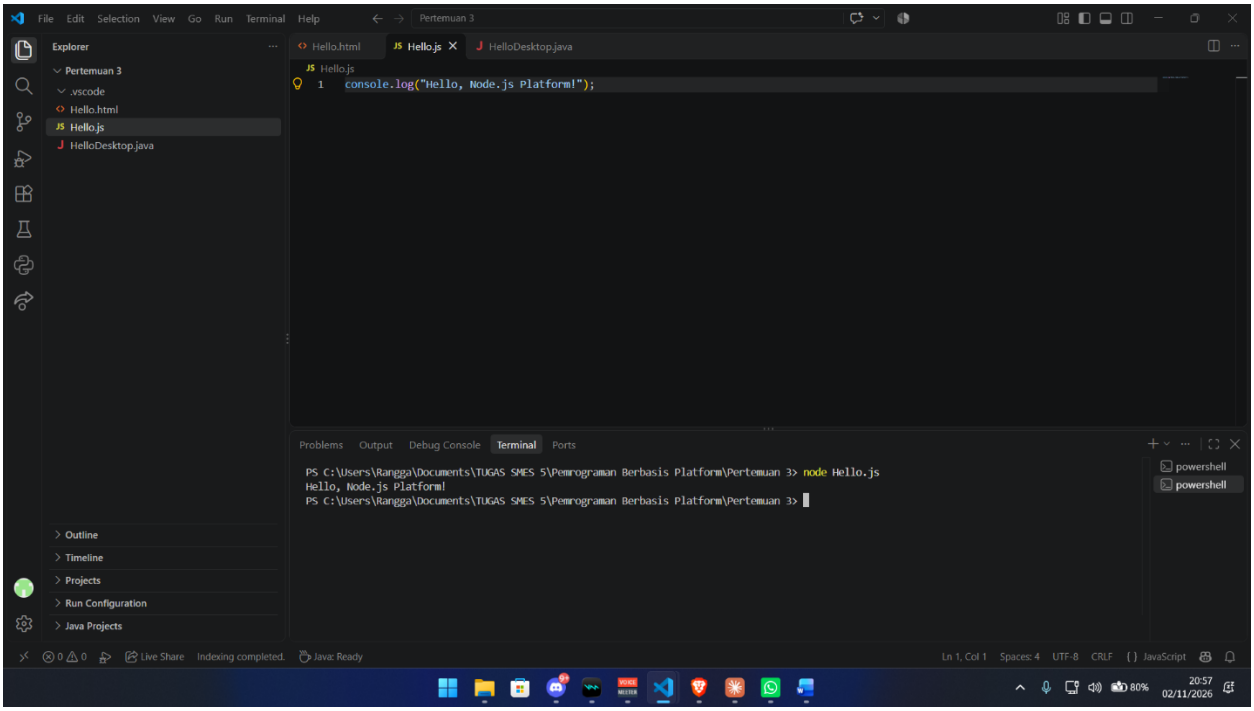
Pengujian	Perintah	Keluaran yang diharapkan	Hasil aktual
Versi Git	git --version	Nomor versi Git tampil	2.56.0.windows.3
Versi Node.js	node -v	Nomor versi berawalan v	v26.13.0
Versi npm	npm -v	Nomor versi npm tampil	v2.0.1
Versi kompilator Java	javac -version	javac diikuti nomor versi	27.0.1.0.3
Halaman web	Buka hello.html	Judul "Halo dari Platform Web" dan identitas praktikan tampil	Hello, Web Platform! Nama : Rangga Alvian Putra Pratama
Node.js	node hello.js	Tiga baris: pesan, versi Node.js, nama sistem operasi	Hello, Node.js Platform!
Java	java HelloDesktop	Dua baris: pesan dan versi Java	Hello, Dekstop Platform!
Git	git log --oneline	Minimal satu commit tercatat	[isi]

A. Platform Web



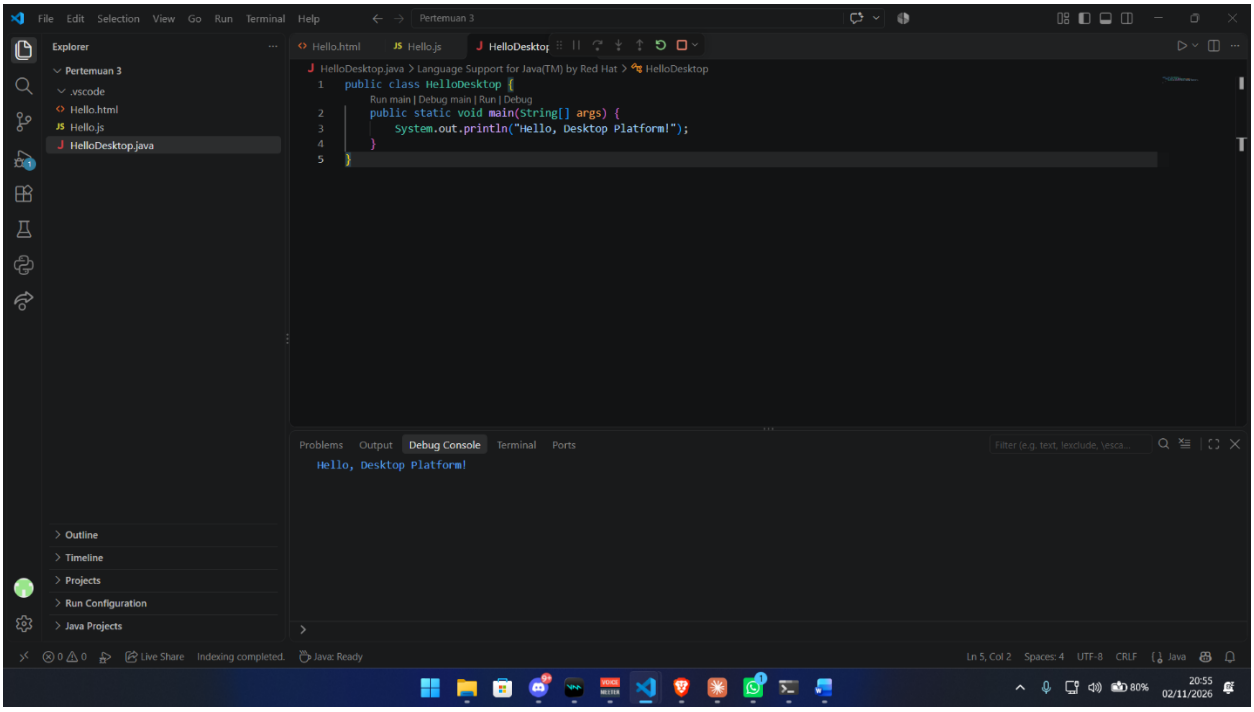
Gambar 4.1. Tampilan hello.html pada peramban.

B. Platform Node.js



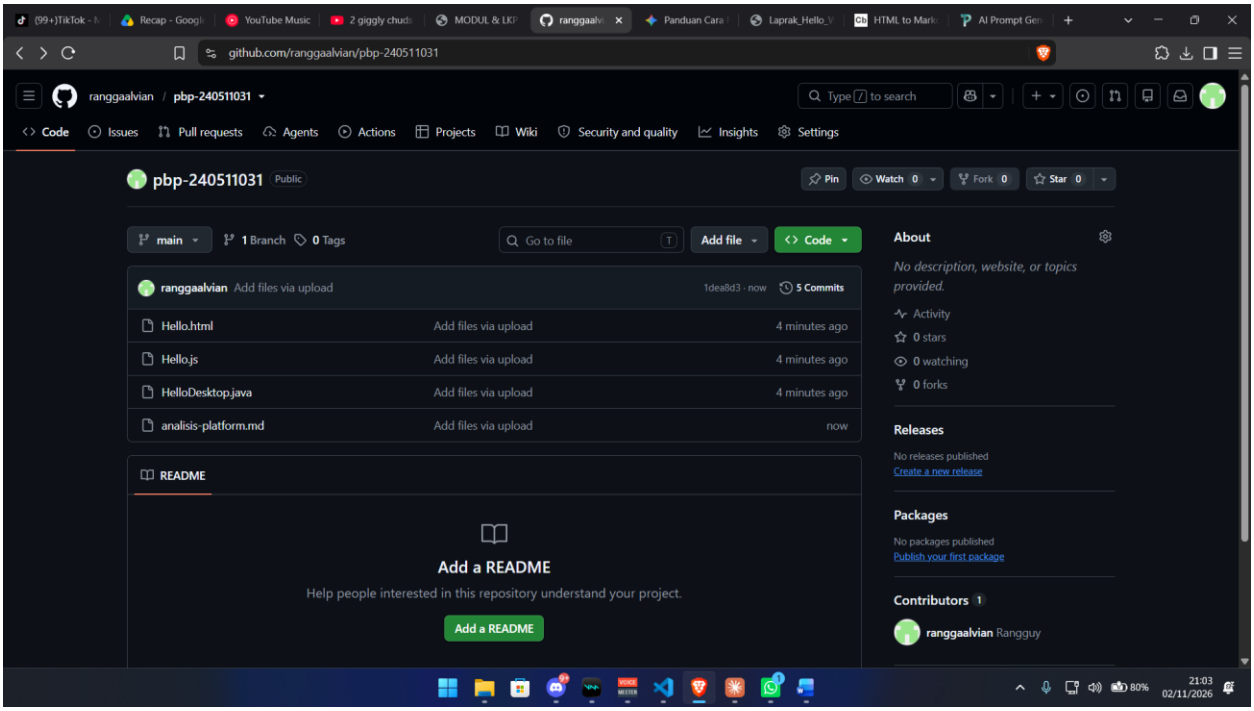
Gambar 4.2. Keluaran node hello.js.

C. Platform Java Desktop



Gambar 4.3. Proses kompilasi dan keluaran java HelloDesktop.

D. Git dan GitHub



Gambar 4.4. Repositori setelah proses push.

4.2 Pembahasan

1. Platform web. Berkas hello.html tidak melewati proses kompilasi oleh pengembang. Peramban mengurai teks HTML, membangun pohon DOM, menerapkan gaya CSS, lalu menggambar hasilnya di layar. Perubahan pada berkas cukup dilihat ulang dengan memuat ulang halaman, sehingga siklus uji berlangsung cepat. Atribut lang="id" dan meta charset ditambahkan agar teks berbahasa Indonesia dibaca dan dilafalkan dengan benar oleh peramban maupun pembaca layar.

2. Platform Node.js. Perintah node hello.js membuat Node.js memuat skrip dan menjalankannya pada mesin V8 tanpa peramban. Penggunaan process.version dan modul os menunjukkan bahwa Node.js memberi akses ke informasi proses dan sistem operasi, kemampuan yang tidak dimiliki skrip yang berjalan di dalam peramban. Sebaliknya, objek document tidak tersedia pada Node.js.

3. Platform Java desktop. Java memerlukan dua tahap: javac memeriksa sintaks dan tipe lalu menghasilkan bytecode, kemudian java meluncurkan JVM untuk menjalankannya. Kesalahan penulisan terdeteksi lebih awal pada tahap kompilasi, berbeda dengan JavaScript yang kesalahannya umumnya baru muncul saat dijalankan. Keharusan nama kelas publik sama dengan nama berkas merupakan aturan bahasa Java dan menjadi sumber galat yang umum bagi pemula.

4. Kontrol versi. Urutan add, commit, dan push memisahkan tiga hal: memilih perubahan, mencatatnya secara lokal, dan membagikannya. Pemisahan ini memungkinkan perubahan ditinjau sebelum dicatat. Keberhasilan push perlu dibuktikan melalui keluaran perintah atau tampilan commit di GitHub, bukan diasumsikan dari keberhasilan perintah lokal.

4.3 Analisis Perbandingan Platform

Tabel 4.2. Perbandingan karakteristik ketiga platform

Aspek	Web (HTML)	Node.js	Java Desktop
Pelaksana kode	Mesin peramban	Mesin V8 pada runtime Node.js	JVM
Langkah sebelum dijalankan	Tidak ada	Tidak ada	Kompilasi dengan javac
Artefak yang dihasilkan	Tidak ada	Tidak ada	Berkas .class
Perintah menjalankan	Membuka berkas di peramban	node hello.js	java HelloDesktop
Akses ke sistem lokal	Terbatas oleh peramban	Luas (berkas, proses, jaringan)	Luas melalui pustaka Java
Deteksi galat	Saat halaman dimuat	Saat dijalankan	Sebagian besar saat kompilasi

Aspek	Web (HTML)	Node.js	Java Desktop
Contoh penggunaan	Situs dan antarmuka web	API server dan alat baris perintah	Aplikasi desktop dan perangkat lunak korporasi

BAB V KESIMPULAN

1. Perangkat pengembangan (Visual Studio Code, Node.js dan npm, JDK, Git) dapat diverifikasi melalui perintah pemeriksaan versi dan program Hello World pada tiga platform.
2. Ketiga platform menjalankan kode dengan cara berbeda: peramban merender dokumen HTML secara langsung, Node.js menjalankan JavaScript pada mesin V8, dan Java dikompilasi menjadi bytecode sebelum dijalankan oleh JVM.
3. Perbedaan tersebut menentukan alur kerja pengembangan, saat galat terdeteksi, serta tingkat akses ke sumber daya sistem.
4. Git dan GitHub memungkinkan riwayat pekerjaan tercatat dan tersimpan di luar komputer lokal, dengan syarat setiap unggahan diverifikasi.

DAFTAR PUSTAKA

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

GitHub Docs. (n.d.). *GitHub documentation*. https://docs.github.com

Lindholm, T., Yellin, F., Bracha, G., Buckley, A., & Smith, D. (2022). *The Java Virtual Machine specification, Java SE 18 edition*. Oracle. https://docs.oracle.com/javase/specs/

MDN Web Docs. (n.d.). *HTML: HyperText Markup Language*. https://developer.mozilla.org/en-US/docs/Web/HTML

Microsoft. (n.d.). *Visual Studio Code documentation*. https://code.visualstudio.com/docs

Node.js. (n.d.). *Node.js documentation*. https://nodejs.org/en/learn

npm Docs. (n.d.). *npm documentation*. https://docs.npmjs.com

Oracle. (n.d.). *Java documentation*. https://docs.oracle.com/en/java/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing, 14*(6), 80–83.